

软件工程大作业架构设计文档

一、系统概述

本系统面向艺术爱好者、学生、研究人员等，旨在为用户提供一个智能美术馆导览平台。包含便捷的美术作品识别与讲解功能，同时满足用户对艺术领域的探索需求。用户可以通过拍摄或上传美术作品的图片，系统利用阿里云 Qwen-VL 模型对图片进行解析，给出作品的描述、创作技巧和情感表达等信息，并支持用户与系统进行对话交流，还能让用户对作品留下评价。此外，用户可以自由探索感兴趣的艺术内容，包括查看全球著名画家信息、艺术作品展示以及博物馆搜索功能。

二、整体架构

系统采用前后端分离的架构，前端负责与用户交互，包括图片选择、预览、上传以及对话、评价界面的展示以及自由搜索功能；后端负责接收前端上传的图片，调用 Qwen-VL 模型进行解析，并处理用户的对话请求。

2.1 前端架构

前端主要由四个页面组成：首页（1-index.html）、图片上传页（2-photo.html）、评价页（3-comment.html）和查看他人评价页（4-view.html）。

首页（1-index.html）：提供系统的入口，显示当前时间，展示一键识图、博物馆 / 美术馆、名人 / 作者等功能按钮，用户点击“一键识图”按钮可跳转到图片上传页，点击“名人/作者”模块，通过搜索功能了解相关作者的信息或者点击“博物馆/美术馆”按钮模块，了解用户感兴趣的艺术馆信息。

图片上传页（2-photo.html）：用户可以选择拍照或从图库中选择图片进行上传，支持图片预览和上传进度显示。上传成功后，显示人物动画并切换到对话界面，用户可以与系统进行对话交流。

评价页（3-comment.html）：用户可以在该页面留下对作品的评价，支持字符计数和提交、跳过、查看他人评价等操作。

查看他人评价页（4-view.html）：用户可以查看他人评价，点赞、回复等。

（由于时间原因未能完整搭建，仅作展示使用）。

2.2 后端架构

后端采用 Python Flask 框架搭建，主要包括两个核心接口：文件上传接口（/upload）和对话接口（/chat）。

文件上传接口（/upload）：接收前端上传的图片，保存到本地，调用 Qwen-VL 模型进行解析，将解析结果保存到本地，并为用户初始化会话。

对话接口（/chat）：接收用户的对话请求，将用户消息添加到会话历史中，调用模型获取回复，将回复添加到会话历史中并返回给前端。

2.3 数据流向

2.3.1 一键拍照模块

用户在前端选择图片并上传。

前端将图片通过 FormData 发送到后端的文件上传接口（/upload）。

后端接收图片，保存到本地，调用 Qwen-VL 模型进行解析，将解析结果保存到本地，并为用户初始化会话。

后端将解析结果返回给前端。

前端显示解析结果，并切换到对话界面。

用户在对话界面输入消息，前端将消息发送到后端的对话接口（/chat）。

后端接收消息，将消息添加到会话历史中，调用模型获取回复，将回复添加到会话历史中并返回给前端。

前端显示回复。

用户可以在评价页留下对作品的评价，前端将评价信息发送到后端进行处理。

2.3.2 博物馆/美术馆模块

博物馆/美术馆信息存储在 museums 数组中，用户在搜索框输入关键词，点击搜索按钮或按下回车键后，执行 performSearch 函数，遍历 museums 数组，筛选出匹配的博物馆/美术馆信息并动态添加到搜索结果区域。数据流向为：

museums 数组 -> performSearch 函数 -> 搜索结果区域。

2.3.3 名人/作者模块

画家数据：画家信息存储在 artists 对象中，页面加载时，默认加载齐白石的信息。当用户点击侧边栏的画家名称时，通过 loadArtist 函数根据画家 ID 从 artists 对象中获取对应画家信息，并动态更新页面内容。数据流向为：

artists 对象 -> loadArtist 函数 -> 页面元素。

搜索数据：用户在搜索框输入关键词，点击搜索按钮或按下回车键后，执行 performSearch 函数，将用户输入的关键词传递给百度百科进行搜索。数据流向为：用户输入 -> 搜索框 -> performSearch 函数 -> 百度百科。

三、关键决策设计

3.1 前后端分离架构

采用前后端分离架构的主要原因是提高系统的可维护性和可扩展性。前端和后端可以独立开发、测试和部署，降低了开发和维护的难度。同时，前后端分离也方便了系统的扩展，例如可以轻松地更换前端框架或后端语言。

3.2 选择阿里云 Qwen-VL 模型

Qwen-VL 是阿里云推出的多模态大模型，具有强大的图像理解和文本生成能力。选择该模型可以准确地解析美术作品的图片，提供详细的描述、创作技巧和情感表达等信息，满足系统的核心需求。

3.3 会话管理

为了实现用户与系统的对话交流，系统采用会话管理机制。后端使用用户的 IP 地址作为简单的用户标识，为每个用户维护一个会话历史，将用户消息和系统回复都添加到会话历史中，以便模型能够根据上下文进行回复。

3.4 进度条设计

在图片上传过程中，前端使用进度条来显示上传进度。通过每秒增加 5% 的方式模拟上传进度，当达到 100% 或上传完成时清除定时器，增强了用户体验。

3.5 文本格式处理

在显示对话消息时，前端对用户输入的消息进行了简单的 Markdown 格式处理，将换行符、标题、加粗和列表等语法转换为 HTML 标签，提高了消息的可读性。

3.6 动画效果设计

在图片上传成功后，前端显示人物动画，增加了系统的趣味性和交互性。通过 CSS 动画实现人物的走进效果，提升了用户体验。

3.7 数据存储方式

画家信息使用对象（artists）存储，每个画家作为一个键值对，包含画家的基本信息、画像、背景图片、描述、代表作品信息和相关链接。这种设计方便根据画家 ID 快速查找和获取对应画家的信息。

博物馆信息使用数组（museums）存储，每个博物馆作为一个对象，包含博物馆的名称、位置、官方网站链接和图片链接。这种设计方便遍历和筛选博物馆信息。

3.8 页面交互设计

侧边栏切换：在画家详情页，侧边栏的画家名称可以点击切换，点击后添加激活状态，并加载对应画家的详情信息。这种设计增强了用户的交互体验，方便用户快速查看不同画家的信息。

搜索功能：在画家详情页和博物馆搜索页都提供了搜索功能，用户可以输入关键词进行搜索。在博物馆搜索页，搜索结果会动态显示在页面上，若未找到相关结果，会显示无结果提示。这种设计方便用户快速查找所需信息。

四、技术选型

4.1 前端技术

HTML/CSS：用于构建页面结构和样式。

JavaScript：用于实现页面的交互逻辑、数据处理和页面的动画效果。

Tailwind CSS：用于快速构建响应式的用户界面。

Font Awesome：用于提供图标库。

4.2 后端技术

Python Flask：用于搭建后端服务器。

阿里云 Qwen-VL 模型：用于图像解析和文本生成。

Base64 编码：用于将图片转换为 Base64 字符串，方便在网络中传输。

JSON：用于前后端数据的传输和存储。

五、总结

本系统采用前后端分离的架构，结合阿里云 Qwen-VL 模型，实现了美术作品的识别与讲解、作者和艺术馆搜索查询的功能。通过会话管理、进度条设计、文本格式处理和动画效果设计等关键决策，提升了系统的用户体验和可维护性。同时，选择合适的技术栈，确保了系统的性能和稳定性。